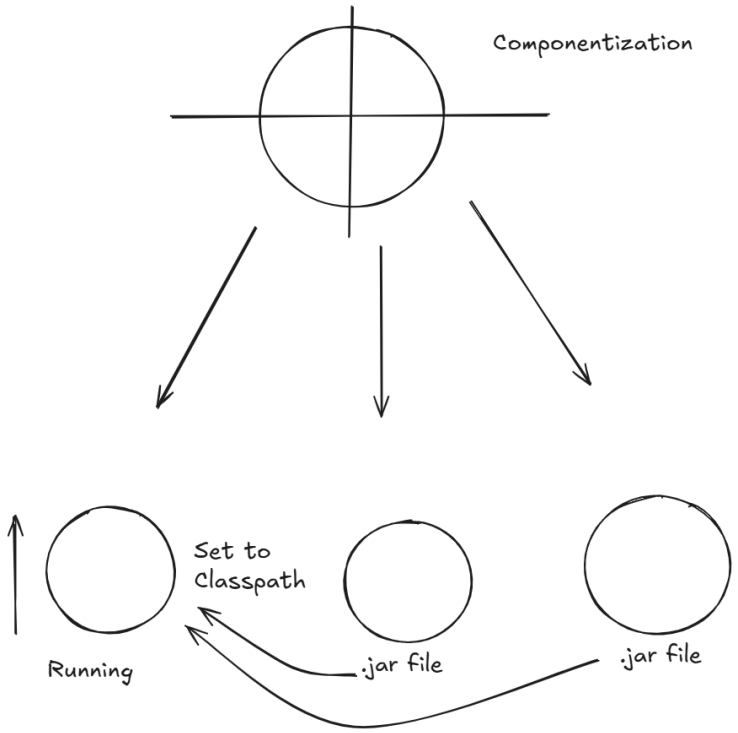
1. Componentization via Services

First we need to understand the meaning of Componentization:

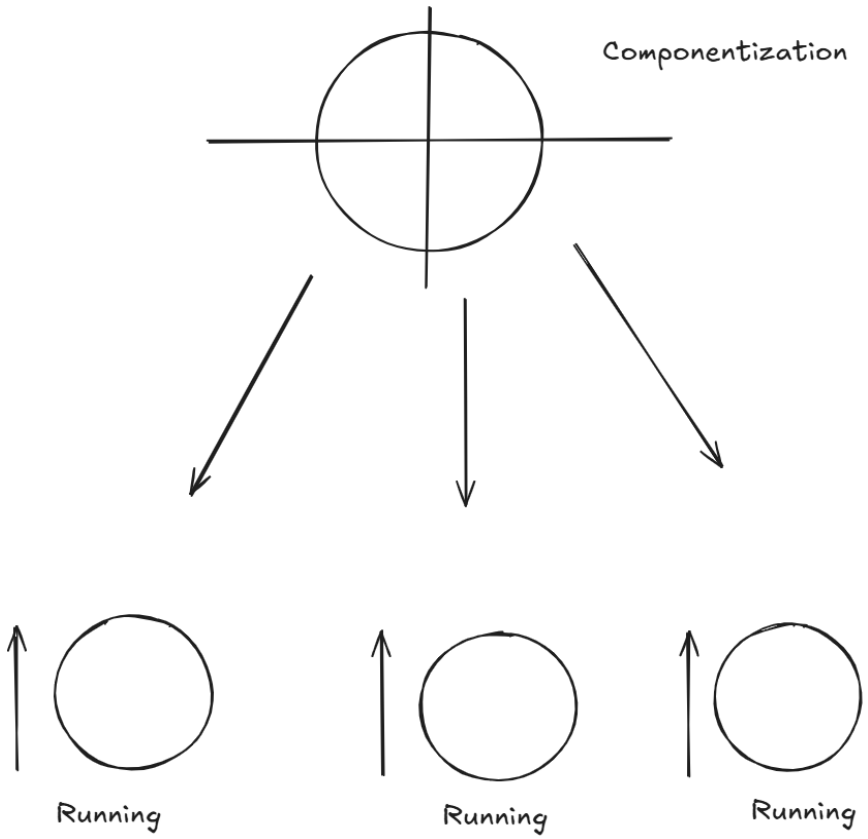
- It means to modularize the big system into smaller maintainable modules which can be managed independently.
- In this way the scalability and flexibility will increase and the system will be easy to maintain.

There is one more concept of Componentization via Library:

- In this approach we will keep one component up and running and the other component will be packaged as JAR files and set to the classpath of the running component.
-
- This approach looks similar with Componentization via Services approach but has some cons
    - The traffic will only hit the running component and it directs to the functionality, the single point of entry.
    - Whenever we have to scale one component, we need to restart the whole system, tightly coupled we have to unnecessarily allocate scaled resources (RAM, Hard Disk, OS) to every component.
    - It is hard to maintain, not flexible, not scalable.



But in Componentization via Services we will run every component as an independent service



2. Organized around business Capabilities

Every service should be organised around business functionalities that means every independent functionality should be divided into separate services.

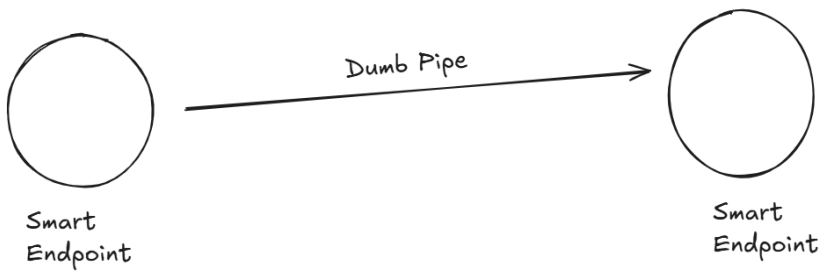
The size is not the measure to determine if it is a microservice or not the size of the microservice should be manageable by the team of approximately 7- 8 members.

If the size is further increased we should create one more microservice and keep common logic in that microservice.

3. What are Smart Endpoints and Dumb Pipes?

It means:

- Endpoints (service) contain the business logic and intelligence.
- The communication layer (pipes) is kept simple and lightweight.



1. Smart Endpoints
   Smart endpoints are services that contain the actual business logic and decision-making.

   Each Service is Responsible for:
   - Processing requests
   - Applying business rules
   - Managing its own data
   - Returning results

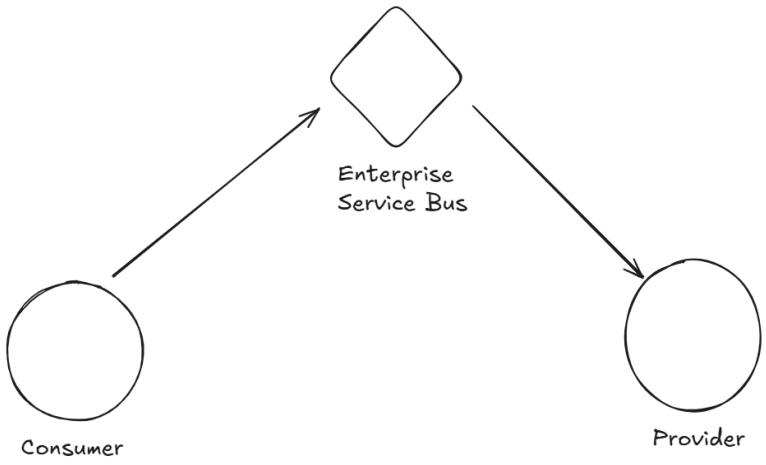
2. Dumb Pipes
   A dumb pipe is a communication mechanism that simply transfers messages without processing them.

   Examples:
   - HTTP
   - Rest API
   - Message queues
   - Event Streams

These systems just move data from one service to another. They do not contain business logic.

In SOA (Service-Oriented Architecture)

It is an architectural style where applications are built using loosely coupled services that communicate over a network.



Components of SOA Communication

1. Service Provider
   The System that offers the service.
2. Service Consumer
   The application that calls the service.
3. Service Registry
   Stores information about available services.
4. Enterprise Service Bus (ESB)
   Communication often goes through an ESB.

Drawback of ESB (Middleware)

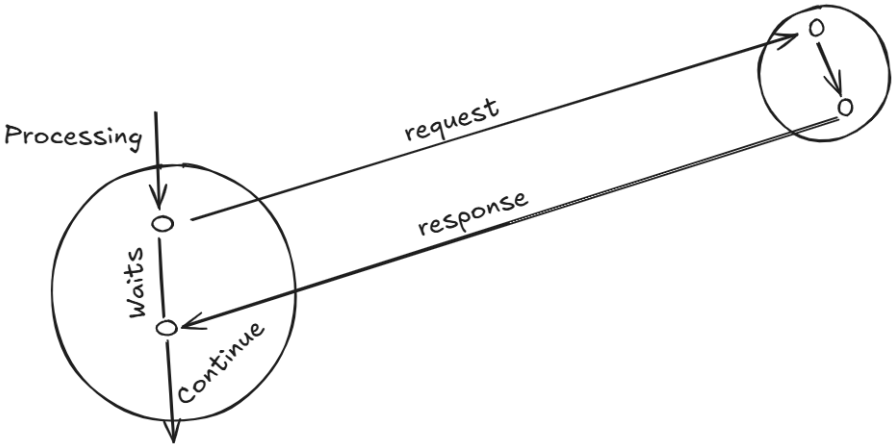
1. Single point of failure
2. Performance bottleneck
3. Complex System
4. Difficult debugging
5. Slower development
6. Scaling Problem

The way of Communication in Microservices:

1. Synchronous
2. Asynchronous

1. Synchronous:

One service sends a request to another service and waits for the response before continuing.



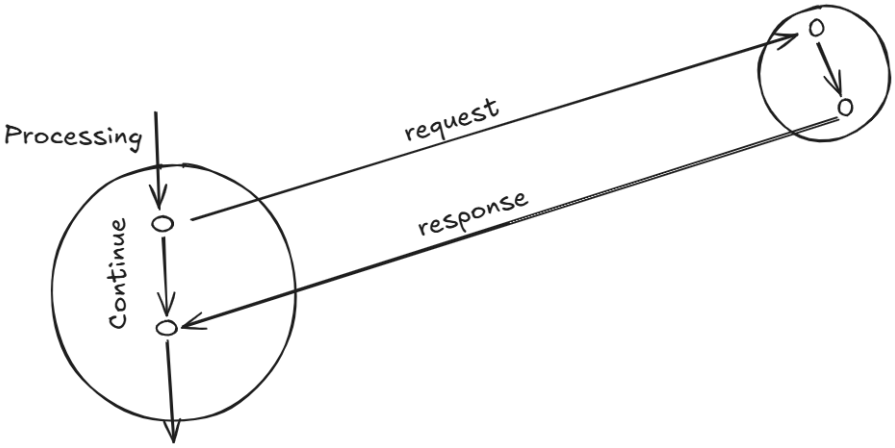
Libraries:

- RestClient
- RestTemplate
- Http Service Client
- FeignClient

RestClient is an alternate of RestTemplate and Http Service Client is alternate of FeignClient Because these are old libraries.

2. Asynchronous:

A service sends a request but does not wait for the response immediately.



2 ways to implement:

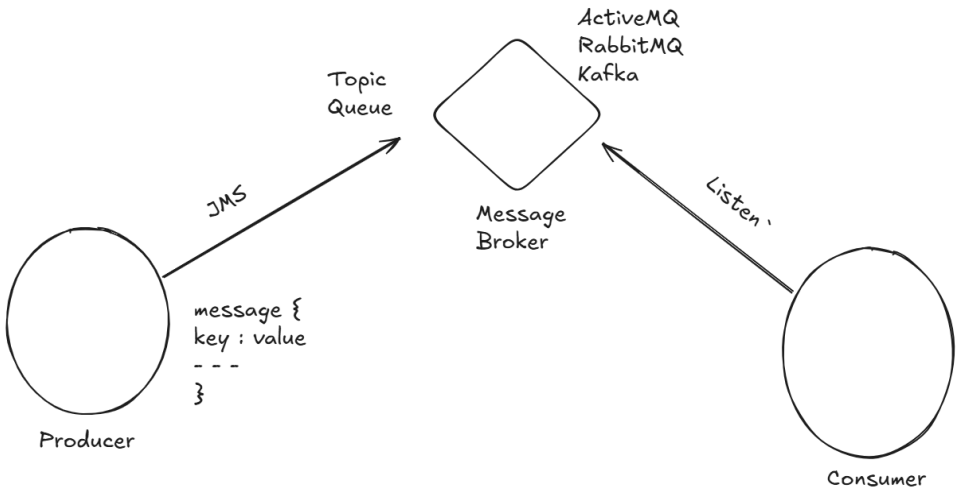
1. Message Brokers
2. Reactive Style of application

1. Message Broker:

In this approach the service sends the Message using JMS API in the request and the Message broker keeps the list of messages in Queues or Topics.

The popular Message Brokers are ActiveMQ, RabbitMQ, Kafka

And the other microservice will listen to the messages and then process and return responses.



What is JMS?

JMS (Java Message Service) is a Java API used for sending, receiving and processing messages between applications asynchronously.

It allows different applications or services to communicate through messages instead of direct API calls.

JMS is part of the Jakarta EE ecosystem.

Basic JMS Architecture:

Components:

1. Producer - sends messages
2. Consumer - receives and processes messages.
3. Message Broker - manages message delivery between producer and consumer.

JMS Messaging Models:

It supports two communication models.

1. Point-to-Point (Queue Model)
    - Messages go to one consumer only.
    - Only one service processes the message.
    - Use Case:
        - Task Processing
        - Background jobs

2. Publish-Subscribe (Topic Model)
    - Message goes to multiple consumers.
    - Use Case:
        - Event-driven systems
        - Notifications

    Failure Scenarios:

    1. Message Retry Scenario
        - Sometimes Message processing fails. The broker can retry message delivery
    2. Dead Letter Queue (DLQ)
        - Isolate problematic messages. Prevents blocking other messages.
    3. Message Ordering Scenario
        - Some systems require messages to be processed in order.
    4. Message duplication Scenario
        - Sometimes the same message may be delivered multiple times.
        - Reasons:
            - consumer crash
            - network issue
            - retry mechanism
    5. Message Persistence Scenario
        - Message should not be lost if the broker crashes.
    6. Backpressure Scenario
        - Sometimes producers send messages faster than consumers can process.
        - Possible Solutions:
            - Rate limiting
            - Scaling consumers
            - Message batching
    7. Message Timeout Scenario
        - Some messages become irrelevant after time. Broker supports TTL (Time To Live)
    8. Event Streaming Scenario
        - Modern systems use brokers for continuous event streams.
    9. Service Failure Scenario
        - If consumer service goes down.

2. Reactive Communication

Reactive communication means services communicate in a non-blocking, asynchronous, event-driven way, where the system reacts to events instead of waiting for responses.

Reactive systems are built on four main principles defined in the Reactive Manifesto:

1. Responsive
2. Resilient
3. Elastic
4. Message-driven

Most reactive systems rely on message-driven communication.

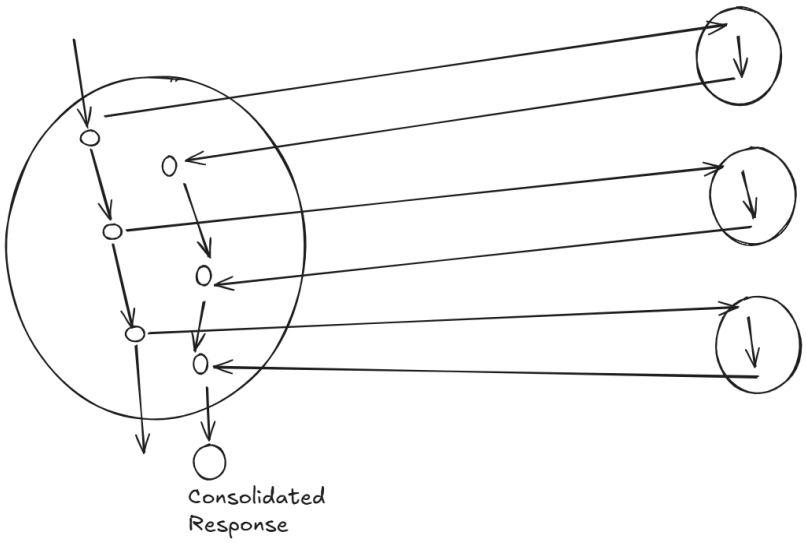
Technologies Used for Reactive Communication:

- Spring WebFlux — WebClient library
- Project Refactor
- Apache Kafka
- RSocket

These tools enable non-blocking data streams.

What is consolidated response handling in reactive systems?

Consolidated response handling is the process of collecting responses from multiple asynchronous services and combining them into a single final response. In a reactive system this is achieved by executing service calls in parallel and merging the results using reactive streams.



4. Design for Failure

When building distributed systems, we assume components like services, networks, or databases can fail at any time, so the system must be designed to continue working or recover automatically.

In distributed architectures, failure can happen due to:

- Network issues
- Service crashes
- Database downtime
- High traffic
- Timeouts

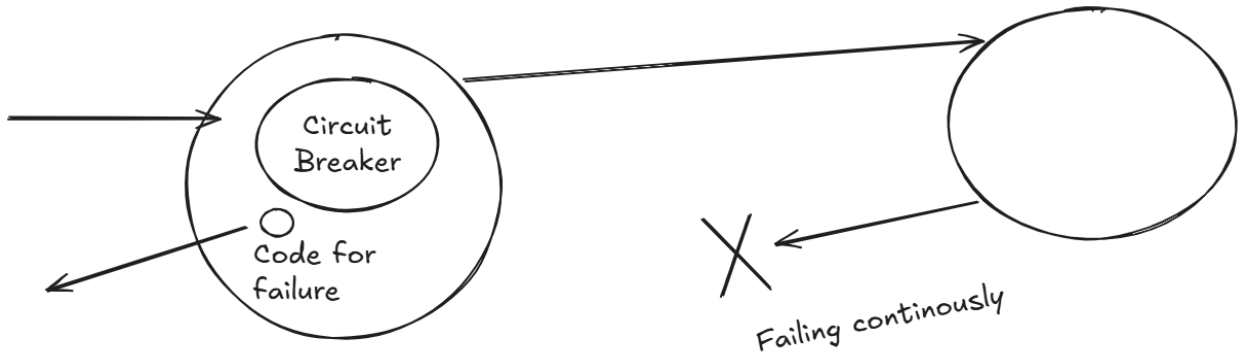
Instead of preventing failure completely, we build systems that tolerate failure.

1. Circuit Breaker:
   When one Microservice send request to other microservice that microservice is continuously failing to respond and traffic is generated that can crash the service

   The failure due to any reason like network issue, resource(CPU, Hard Disk, RAM) full, Service crashes, etc.

   So Circuit Breaker stop calling failed service and send response back to the sender as a failure

After time Circuit Breaker check whether the service is working or not
If it is working then connects else cut the communication and send response as failure



The Library in Spring Boot : Resilience4j

2. Real-Time Monitoring and Alerting:

Real-time monitoring means continuously tracking the health, performance, and behaviour of a system as it runs.

It collects metrics such as:

- CPU usage
- Memory usage
- Request latency
- Error rate
- Service availability

Definition:

Real-time monitoring is the process of collecting and analyzing system metrics, logs, and events continuously to understand system performance and detect problems immediately.

Tools:

- Prometheus and Grafana
- Nagios
- Jaeger
- Zipkin

Alerting:

Alerting is the automated process of sending notifications when monitoring systems detect abnormal behaviour of threshold violations.

Examples:

- CPU usage > 90%
- API response time > 2 sec
- Service downtime
- Error rate spikes

Spring Boot Dependency: spring-boot-starter-actuator

3. End-to-End Logging:

   Consider an example if we are having multiple services in the process of making transactions, and if the transaction is failed in the middle means money is deducted but not received at the end.

   Every Service is maintaining logs. One service in the middle experiences more traffic then we scale by creating instances. In these instances we also keep a log file and this transaction is made by this instance if the traffic is getting less we have to delete the instance along with the generated log file this process is automatic.

   So the transaction history is gone from the middle, which is not the correct way to handle such a situation. We have to keep the logs at least 20 years for safety reasons and as per the regulations.

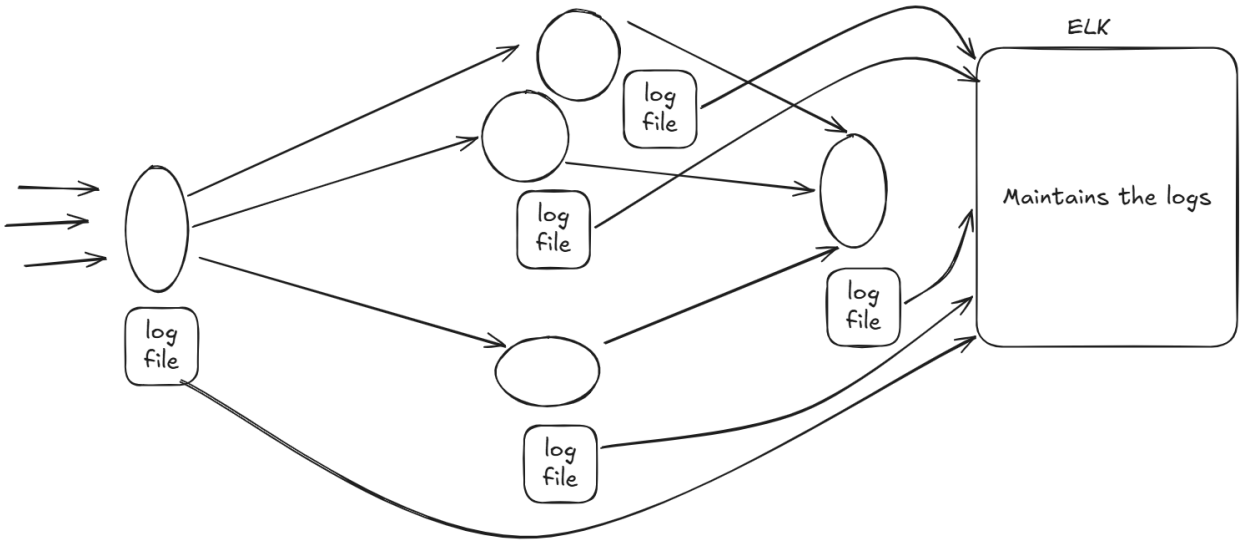
   For that we can use ELK (Elasticsearch Logstash Kibana)

   Elasticsearch:

   It organizes data into indices containing documents stored as JSON. It distributes these across multiple nodes in a cluster, supporting automatic sharding and replication for high availability.

   Logstash:

   It acts as a data pipeline with three main stages: input, filter, and output. Inputs collect data from sources such as logs, metrics, or message queues. Filters transform, parse, or enrich this data.

Kibana:

It provides a unified interface for creating visualizations such as bar and line charts, pie graphs, time-series plots, and geographics maps.